

Ire's Deliverables

What Ire owns, what is finished, what is still to do, and the number that says each item is done.

Ire owns the data foundation and the evidence for detection: the simulator that generates the bank's customers and its fraud, the twelve measurements taken on every payment, the trained model, and every evaluation that says how well the rules and the model work. Chidera wires these into the live system; Ire's job is to make sure what goes in is realistic and what comes out is measured honestly.

Where things stand

AREA	MEASURE	TARGET	NOW	STATUS
Simulator	Fraud rate against Nigeria's interbank settlement body (NIBSS)	≈ 0.30%	0.296%	MET
Simulator	Fraud incidents available to test on	≥ 600	604	MET
Simulator	Measurements that give fraud away on their own	0 over 15%	0 · worst 5.6%	MET
Simulator	Normal behaviour checked against published Nigerian figures	key shares	fraud rate only	PARTLY
Measurements	Same twelve numbers in training and live	identical	identical, tested	MET
Model	Honesty of its probability (stated ÷ actual, where alerts are raised)	≥ 0.90	0.92	MET
Model	Calibration error where alerts are raised	≤ 0.04	0.032	MET
Evaluation	Account takeover caught, never seen in training	≥ 0.70	0.777	MET
Evaluation	Mule fan-out caught, never seen in training	—	0.909	UP FROM 0.649
Evaluation	Card testing caught, never seen in training	≥ 0.90	0.990	MET
Evaluation	Every rule measured on its own	all six	three	PARTLY
Validation	Public dataset with invented labels (PaySim)	run	done	MET
Validation	Public dataset with real fraud labels (IEEE-CIS)	run	blocked	BLOCKED

"Never seen in training" means that kind of fraud was hidden from the model while it learned, with the team held to 120 alerts a day. Average across the three: **0.892** (decision D67a). On its own the model scores 0.403, below a single raw measurement; it is useful only alongside the rules, and must always be described that way.

Finished — Ire presents these at the defence

- **The simulator.** Customers of four kinds (ordinary, salary earner, small business, trader) who own one to three accounts, keep habits, pay known and new people. Three kinds of fraud generated as criminal processes, not as rule thresholds: account takeover, mule fan-out, card testing. Identities now come from the seed, so the same seed rebuilds the same bank (D68).
- **The twelve measurements,** one shared package for training and live, with a test proving they match. Version 1.1.0: "does not apply" is marked -1, ratios are capped at 100 (D64).
- **The calibrated model:** gradient-boosted trees, trained on older data and adjusted on newer data it never saw, so its probability means what it says (D60–D61).
- **The held-out evaluation:** six approaches compared on identical data at the same alert budget, with 95% ranges on every figure (D59, D63, D65, D67a).
- **The self-audit** that caught eight "giveaway" measurements in our own data and led to the realistic population (D57), and the reproduction of the earlier dataset's result (D56).
- **PaySim validation:** 65% of fraud value caught while catching 18% of fraud transactions; six of our twelve measurements cannot fire on PaySim, which we state rather than hide.

Still to do

1 Check "normal" against published Nigerian figures NOT STARTED

Why	The most serious objection to the project is that our normal customers are invented. This turns "we made it up" into "we made it up and checked its shape".
What	Take published aggregates (NIBSS annual fraud reports, Central Bank payment statistics) and compare the simulator's shares side by side: channel mix, average transfer value, interbank (NIP) versus in-bank share, fraud loss by channel.
Done when	A table of our share against the published share for each; any gap wider than a third of the published value is either corrected in the simulator or explained in writing.

2 Run the real-label validation (IEEE-CIS) BLOCKED ON A PERSON

Why	The only test short of a real bank with genuine fraud outcomes (about 590,000 transactions, 3.5% fraud).
What	Sign in to Kaggle, complete phone verification, accept the competition rules at kaggle.com/competitions/ieee-fraud-detection/rules , put <code>train_transaction.csv</code> in <code>ml/data/public/</code> , then run <code>python ml/validate_public.py --dataset ieee</code> . The script is written and ready.
Done when	Results file written; precision-recall area, calibration error and recall at the alert budget reported, with the caveat that it checks method, not our measurements (its columns are anonymised).

3 Measure the last three rules one at a time 3 OF 6 DONE

Why	The Blueprint requires every rule to be tested on its own and dropped if the data does not support it. Card testing (right 99.8% of the time), the payment-burst rule (D67) and the familiar-payee rule (D64) are done.
What	For "saved payee" (lowers risk): how often it fires on fraud, which would hide it. For "sanctioned destination" and "known mule destination" (override everything): seed test lists and confirm they fire on every listed destination and nowhere else.
Done when	Each of the three has a firing rate, a share of fraud it wrongly lowers or correctly catches, and a keep-or-drop recommendation logged in the decision log.

4 Give the demo bank a longer memory CAUSE FOUND

Why	The payment-burst rule now fires only on payments to a destination new to the bank. The demo bank remembers about 3 days, so a third of its payments look "new" and the rule fires about 50 times a day, 5 of them on fraud; in the 30-day training data it fires about 34 times a day (D67d).
What	Lengthen the demo profile's history (in <code>simulator/riskradar_sim/generate.py</code>) from 4 days towards 21–30, and measure how long the demo rebuild then takes.
Done when	On a rebuilt demo the rule fires at most about 35 times a day, the whole system stays within 120 alerts a day, and the rebuild time is written in the README.

5 Support the team's alert-budget decision EVIDENCE READY

Why	With the new rule, a smaller budget cuts false alarms sharply: 120 a day gives about 75 false alarms, 75 a day about 33, 60 a day about 20. The cost is account takeover never seen before: 0.777 at 120, 0.703 at 75, 0.658 at 60 (D67c).
What	Walk the team through the trade-off; once they choose, re-run <code>ml/evaluate_system.py</code> at the chosen budget and confirm the figures quoted in the Blueprint.
Done when	The team's choice is logged as a decision and every document quotes the figures for that budget.

Rules Ire works under

- **Detection never reads the simulator.** Rules and model see only the payment and its measurements, never a generator setting; a test enforces this.
- **Publish the held-out number**, never the flattering one from fraud the model has already seen.
- **Count incidents, not transactions**, and never report accuracy: at 0.3% fraud, "never fraud" scores 99.7%.
- **Never train on someone else's column** named risk, score, rate or fraud unless the code that made it can be read (D10f).
- **Log every change** to what the numbers mean in the decision log, with its evidence, before quoting it.